



DATA STRUCTURES

CONTENTS



2. List and List ADT

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. This is the second module of the paper and we will be talking about list and list ADT – a basic data structure. You will be studying list ADT and the different operations associated with lists.

Learning Objectives

The learning objectives of the introductory module are as follows:

- To describe the list and its uses
- To understand the list ADT and know about the different operations associated with the list ADT
- To understand the different types of list

2.1 Every Day Lists

Before we go further let us look at the use of lists in everyday life. Today we use lists in a more sophisticated manner by having it on a mobile phone but even before this we used lists in a conventional manner by either having a mental list or having a grocery list written down. We have jobs to do list, list of assignments to be done, Dean's list of students selected for placement or any other situation where we need to specify some sequential order of elements. May be you can think of other lists you do use in your everyday life.

2.2 Properties of Lists

Let us first ask some questions about lists.

Can lists have a single element? Can lists have no element? Or can there be a list of lists- the answer to all these questions is yes. So let us look at some properties of lists. Remember we will be looking at the list as an abstract data type. There are some properties associated with any list ADT. These properties include the list has to be homogenous, it is of finite length and it has a sequence of elements . This essentially means that all the elements of the list are of the same type, that it can have any length but we are dealing with a finite length and then there is some sequence associated with the elements – so that is the definition of list

Let us look at an example. Consider a room with 40 seats in fixed positions. Now desks are wasted if there are less than 40 people and there are not enough desks if there are more than 40 people. The array can be considered as a room where each desk is an array location. Suppose we have some persons in the room who are seated at the desks ordered alphabetically according to their names. We want to add a new person but we want to maintain the alphabetical order – this means we need to shift some people. If we remove a person from the middle of the sequence again we need to shift some people.

Now look at the example Figure 2.1 given below: Assume the people are seated in locations in alphabetic order.

Anna	Bhuvana	Chander	Eve	Gowri	Hema	Kalyani	Leela	Mani
1	2	3	4	5	6	7	8	9

Figure 2.1. An alphabetically ordered list

Now suppose Deepak enters the room. He has to be seated between Chander and Eve in order to maintain the alphabetic order and so all people from Eve to Mani have to shift one place to the right to make room for Deepak. Similarly if one person leaves the room and we do not want a vacant location, then all people after that has to shift one place to the left.

Now let us continue with the definition of an ADT list. Except for the first and last items, each item has a unique predecessor and a unique successor. However, head or front do not have a predecessor and likewise tail or end do not have a successor Figure 2.2. In other words a **Linear List** is a list in which each element has a unique successor.

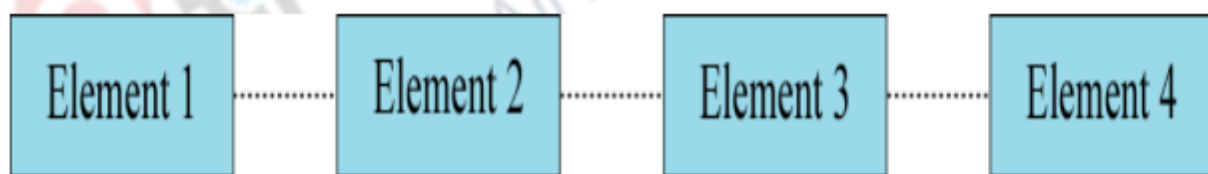
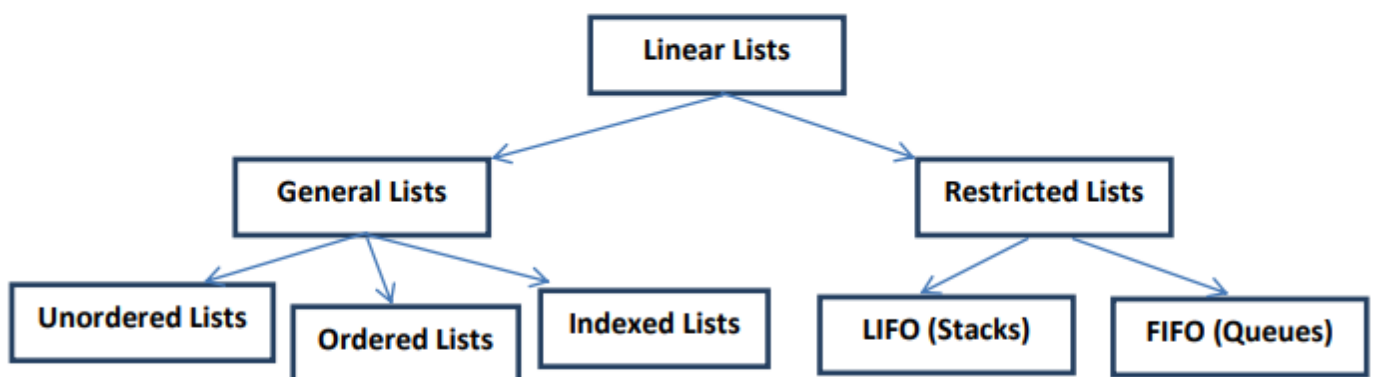


Figure 2.2. A List

2.2 Categories of Linear Lists



Data can be inserted and deleted anywhere. Data can only be inserted and deleted at the ends of the list.

Figure 2.3. Categorization of Lists

Figure 2.3 shows one categorization of lists. The major categories of lists are general lists and restricted lists. In general list data can be inserted to and deleted

from anywhere in the list. Under general lists there are three categories – unordered list with the elements in the list not being in any particular order while in ordered lists – the order of the elements in the list are maintained during insertion and deletion, and indexed lists where insertion and deletion are associated with index into the list. In addition to general lists we have what is called as restricted list. In the case of restricted lists the restriction is on where you can add data. In the first case – you have the FIFO – first in first out type of lists called as stacks – that is the insertion and deletion is allowed only at one designated end of the list. The second case of the restricted lists is the LIFO – the last in first out list called as queue where you insert into one end and delete from the other end.

We begin with a discussion of the basic list operations. These operations include insertion, deletion, retrieval and traversal. In fact these operations are the basic operations associated with any data structure. Now let us look at the operations that we describe for the list ADT. These include constructing an empty list, checking whether the list is empty, finding out the number of items in a list and finally Inserting/Adding an element/item into the list. Another important operation associated with any data structure is the delete operation. There are many variations to the delete operation. These include deleting/removing an element/item from the list, removing the item at a given position in the list, removing all the items from the list, or retrieving (getting) an item (without removing) at a given position in the list. Another operation is traversing (iterating through) the list. In addition some other operations include modifying, outputting, searching for a specific value, copying, saving or rearranging the elements of the list.

2.4 The List ADT

The List ADT models a sequence of positions storing arbitrary objects – establishes a before/after relation between positions. This is an important property associated with lists. A list List of size N can be represented as $A_0, A_1, \dots, A_{N-1}$ where each element A_k has a unique position k in the list. In addition remember that the elements of a list can be arbitrarily complex.

2.4.1 Operations on Lists

Generic operations associated with lists are

- **size()** – this is a method which gives the size of the list and
- **isEmpty()** – is a Boolean method which gives a value of true if the list is empty.

Other operations are **insert**, **insert(X,k)**, **Delete(X)**, **find(X)**, **findKth(k)**, **printList()**, **makeEmpty** and **remove**.

- Assume the Example list is as given below:

the elements of a list are

34, 12, 52, 16, 12

- **insert**–inserts an element to a list
- **insert(X,k)**– which inserts element X at kth position of the list.
- **insert(x,3)** ® 34, 12, 52, x, 16, 12
- **Insert (20, 3)** à 34, 12, 52, 20, 16, 12
- **Delete(X)** which removes the item/element X from the list.
- **Delete (52)** à 34, 12, 20, 16, 12
- **find(X)** – finds out the location of the element X is in the list
- **find(52)** ® 3
- **findKth(k)** – finds the element X at the kth position of the list while
- **FindKth (2)** à 20
- **printList()** – prints the contents of the entire list
- **makeEmpty**– creates an empty list
- **remove**: delete an element from the list

2.4.2 Designing a List ADT

We must remember that the LIST ADT having a different set of operations is a different ADT. However, whatever the purpose for which we are designing the List ADT, it must have the following methods associated with it

Constructor

isEmpty()

insert()

delete()

display()

Implementation of the ADT involves defining representing the data and defining

the operations from the design phase.

These are some of the Accessor methods associated with lists.

first(), last(), prev(p), next(p)

These are some of the Update methods associated with Lists. Here **p** stands for position and **e** for element.

replace(p, e) – element e replaces original element at position p

remove(p)– removes element at position p

insertBefore(p, e)– inserts an element e before position p

insertAfter(p, e)– inserts an element e after position p

insertFirst(e)– inserts element e as first element of list

insertLast(e)– inserts element e as last element of list

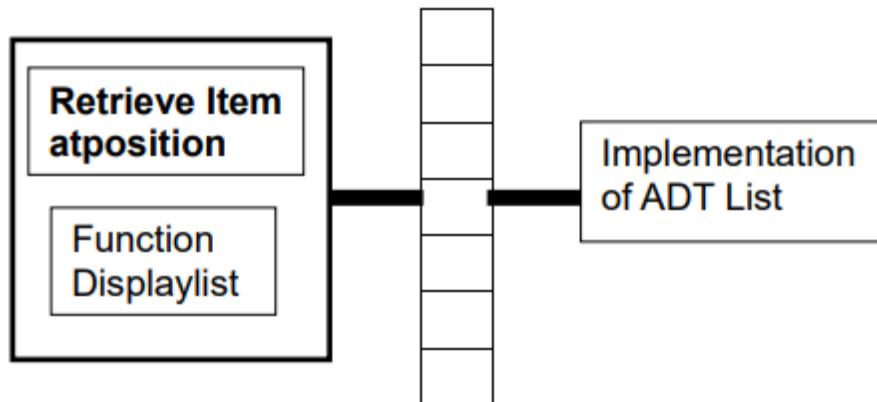
2.5 Position ADT

Position itself can be an ADT. The Position ADT models the notion of place within a data structure where a single object/element is stored. It gives a unified view of diverse ways of storing data.

element(p): returns the element stored at the position

2.6 The ADT List and its Various Forms

Figure 2.4 shows the wall perspective of an ADT where the interface is what is seen by the user of the ADT while the implementation details are hidden from the user of



Wall of ADT operations

Figure 2.4.The wall between *displayList* and *implementation of the ADT list*

the ADT. In the figure the wall of ADT operations separates the display list function, and the retrieval operation for example from the implementation details of the list. As already discussed operations common to *all* categories of lists shown in Figure 2.3.

Removing elements in various ways

Checking the status of the list (**isEmpty**, **size**)

Iterating through the elements in the list

The key differences between the general list types involve the way elements are **added** to an **ordered** list, an **unordered** list and to an **indexed** list.

Before we go to describe the three types of lists let us summarize the different operations on lists (Figure 2.5). Please note that these operation definitions are applicable to all types of lists.

Operation	Description
contains	Determines if particular element is in the list
isEmpty	Determines whether the list is empty
size	Determines number of elements in the list
iterator	Returns an iterator for the list's elements
toString	Returns a string representation of the list
removeFirst	Removes the first element from the list
removeLast	Removes the last element from the list
remove	Removes a particular element from the list
first	Examines the element at the front of the list
last	Examines the element at the rear of the list

Figure 2.5. Common Operations of Lists

Most of the operations in Figure 2.5 have already been discussed. **contains** is an operator that determines if an element is in the list while **iterator** is an operation that returns an iterator for the elements of the list. Another common operator is **toString** that represents a string representation of the list.

As already discussed a list is a linear collection, which is flexible where adding and removing elements from a list does not have to happen at one end or the other.

2.7 Three Types of General Lists

In this section we will examine three types of list collections namely ordered lists, unordered lists and indexed lists. Please note the same operations associated with general lists have been used with appropriate modifications when we define these operations for the three types of general lists.

2.7.1 Ordered Lists

2.7.1.1 Ordered list:

The elements of an ordered list are ordered by some inherent characteristic of the elements. In other words, the elements themselves determine where they are stored in the list. Examples of such lists are very common in our everyday life and for many applications.

Examples:Names in alphabetical order, Numeric scores in ascending order.As we have already discussed it is the add operation that is defined differently for ordered lists.

Operation	Description
add	Adds an element to the list (in the correct place)

2.7.1.2 Insertion in a general linear list with ordered data

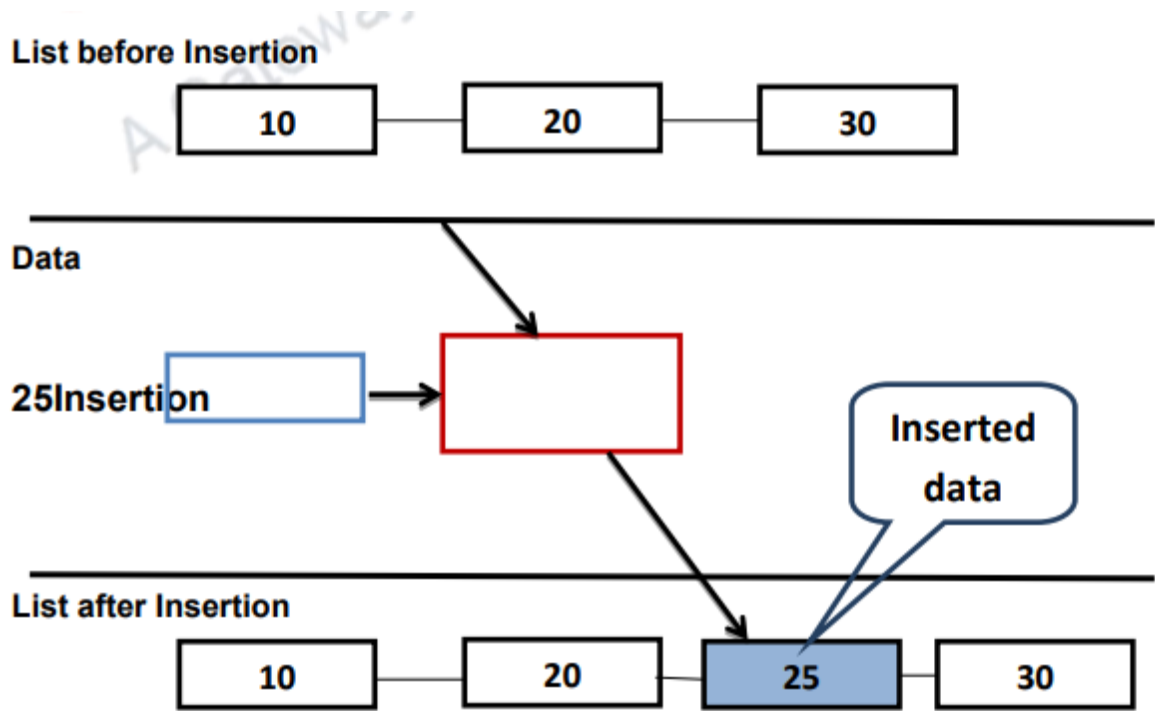


Figure 2.6 Insertion into an Ordered List

Figure 2.6 explains the insertion of data 25 into an ordered list. As explained earlier the position of 25 is decided by the elements already present in the list. Therefore the ADT **sorted** or **ordered** list maintains items in sorted order and inserts and deletes items by their values, not their positions. Figure 2.7 explains how 58 needs to be inserted at position 5.

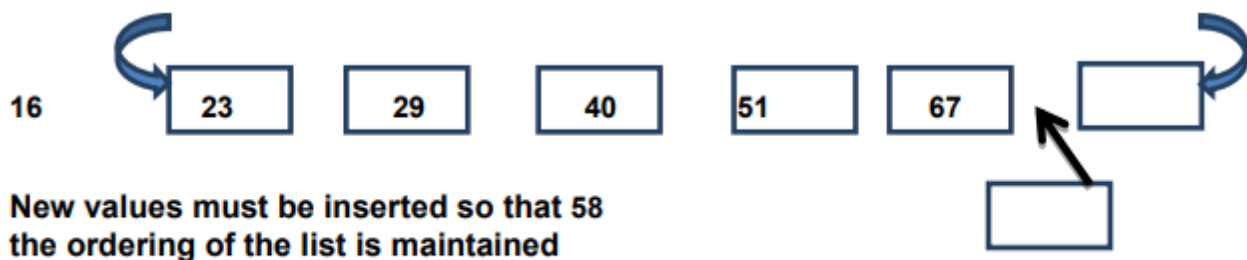


Figure 2.7 Insertion Into Sorted /Ordered List

2.7.1.3 List Search and Retrieval from a List

A list search is used to locate data in a list and is especially important activity for ordered lists. In order to insert data, we need to know the logical predecessor of the new data. Similarly to delete data, we need to find the node to be deleted and identify its logical predecessor. Again to retrieve data from a list, we need to search the list and find the data.

We must use a sequential search because there is no physical relationship among the nodes. The classic sequential search returns the location of an element when it is found and the address of the last element when it is not found. Because the list is ordered, we need to return the location of the element when it is found and the location where it should be placed when it is not found.

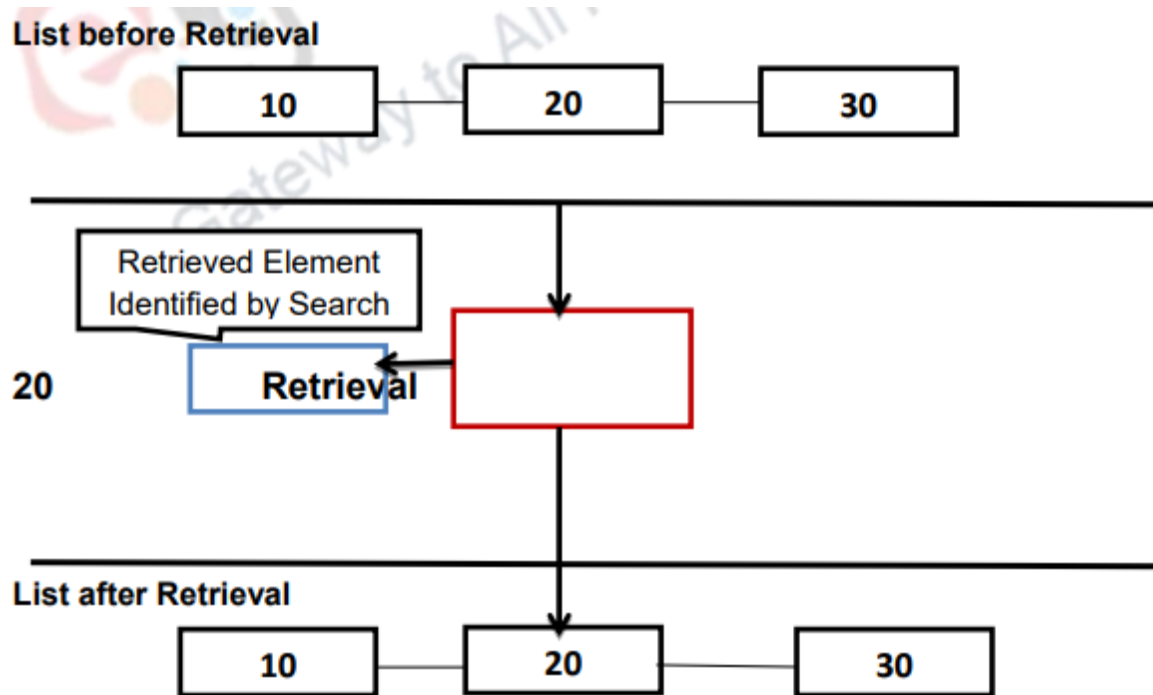


Figure 2.6 Retrieval from an Ordered List

2.7.1.4 List ADT – Retrieving First and Next Element

Now let us look at a simple operation associated with lists- Retrieving the first element from a list L (Figure 2.7). Let us define this as a Boolean function with the first element being returned. There are no preconditions required for calling this function. If the list is empty there will be no postcondition. In case list is not empty the variable **elem** contains the first element of the list. The Boolean function will return true if and only if there is at least one element in L.

```
bool L.first(ListElementType&elem)
```

Precondition: None

Postcondition: If the list is empty, none. Otherwise, the variable elem contains the first item in L; the "next" item to be returned is the second in L.

Return: true if and only if there is at least one element in L

Figure 2.7 Boolean Function First

Similarly let us look at a simple operation associated with lists- Retrieving the

next element from a list L (Figure 2.8). Let us define this as a Boolean function with the next element being returned. The Precondition for this function is that the first operation should have been called at least once. If there is no next element or in other words the list contains only one element there will be no postcondition. In case list is contains more than one element the variable **elem** contains the next element of the list. The Boolean function will return true if and only if there is next item in the list L.

```
bool L.next(ListElementType&elem)
```

Precondition: The “first” operation has been called at least once.

Postcondition: Variable elem contains the next item in L, if there is one, and the next counter advances by one; if there is no next element, none.

Return: true if and only if there is a next item.

Figure 2.8 Boolean Function Next

2.7.1.5 List ADT – Traversing the List

Traversal is an operation in which all elements in the list are processed sequentially, one by one. The process of accessing each item in the list can be defined in terms of two other operations: accessing the first element in a list and accessing the next element in the list. These are the two operations described in section 2.7.1.4. For traversing a list, we need a **walking pointer** that moves from node to node (Figure 2.9)

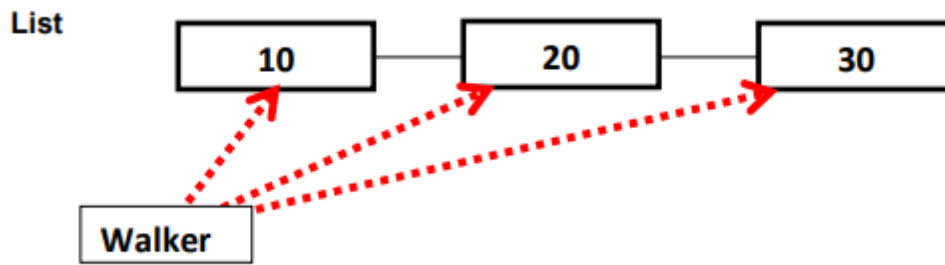


Figure 2.9 Traversal of a Linear List

2.7.2 Unordered Lists

Unordered list are lists where the order of the elements in the list is **not** based on any characteristic of the elements, but is determined by the **user** of the list. A new element can be put on the front of the list, or on the rear of the list or or after a particular element already in the list.

Examples: shopping list, to-do list, ...

As we have already discussed it is the add operation that is defined in three different ways for unordered lists

Operation	Description
addToFront	Adds an element to the front of the list
addToRear	Adds an element to the rear of the list
addAfter	Adds an element after a particular element already in the list

Figure 2.10 Operations Particular to Unordered Lists

2.7.3 Indexed Lists

Operation	Description
add	Adds an element at a particular index in the list
set	Sets the element at a particular index in the list
get	Returns a reference to the element at the specified index
indexOf	Returns the index of the specified element
remove	Removes and returns the element at a particular index

Figure 2.11 Operations Particular to an Indexed List

*Indexed lists are lists where elements are referenced by their numeric position in the list, called its **index**. It is the position in the list that is important, and the user can determine the order in which the items go into the list. Every time the list changes, the position (index) of an element may change.*

Example: current first-place holder in a race.

There are a number of operations particular to indexed list (Figure 2.11). The **add** operation is defined as adding an element at a particular index. We can also defined **set** (setting the element at a particular index, or **get**(returning a reference to the element at the specified index. **IndexOf** will return the index of a specified element while **remove** will remove an element at a particular index and return the element.

2.8 Summary

- Explained the concept of Lists
- Discussed the different operations that can be carried out with lists
- Outlined the different types of lists

you can view video on List and List ADT



Web Links

1. [en.wikipedia.org/wiki/List_\(abstract_data_type\)](https://en.wikipedia.org/wiki/List_(abstract_data_type))
2. www.cs.cmu.edu/~tcortina/15110sp12/Unit06PtA.pdf
3. www.csee.umbc.edu/courses/undergraduate/341/.../Lists/List1.html
4. www.doc.ic.ac.uk/~ar3/lectures/ProgrammingII/.../Lecture2PrintOut.pdf
5. <http://cs.boisestate.edu/~cs221/slides/Lists.ppt>

Supporting & Reference Materials

1. Carrano and Henry, “Data Structures and Problem Solving with C++: Walls and Mirrors”, Pearson; 6 edition, 2012
2. Mark Allen Weiss, “Data Structures and Algorithm Analysis in Java”, Pearson; 3rd Edition, 2011.
3. Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, “Data Structures and Algorithms in Java”, Wiley; 6 edition, 2014